

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:50

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I Bodda Yamini (192525146) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

B. Yal...

COURSE CODE:- CSA1401

COURSE:- Compiler Design

NAME:- Bodda Yamini

Reg NO:- 192525146

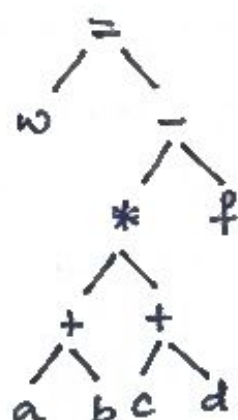
Dept :- AI & ML

CO1 Assessment Tool 1

Question 1:- Compilation Phases for $w = (a+b)*(c+d) - f$

Answer:-

- Lexical Analysis (Tokens) :- $w, =, (, a, +, b,), *, (, c, +, d,), -, f$
- Syntax Analysis (Parse Tree):-



- Intermediate Code (Three-Address Code):-

- $t_1 = a + b$
- $t_2 = c + d$
- $t_3 = t_1 * t_2$
- $t_4 = t_3 - f$
- $w = t_4$

- Code Optimization:-

Reuses temporary variables and evaluates register assignments directly.

- Target Code (Assembly Representation):-

- LOAD R_1, a
- ADD R_1, b
- LOAD R_2, c
- ADD R_2, d
- MUL R_1, R_2
- SUB R_1, f
- STORE w, R_1

Question 2:- Tokenization of float result = index ---- value+++
total;

Answer:-

• Ambiguity:-

---- can be split as -- (decrement) followed by
-- (decrement), or -- followed by - (minus)

• Lex Behavior (Longest Match Rule):-

Lex consumes the longest valid prefix first, recognizing
the first - as a post decrement operator.

• Refined Rule :-

Separate consecutive operators using spaces or explicit
precedence rules to prevent bad token groupings

* Expression re-written as:-

index -- -- value + ++ total;

• Tokens Produced:-

float, result, =, index, --, -, -, value, +, ++,

total, ;

Question 3:- Token Count and Lexical Errors.

Answers:-

⇒ Total Token Count:-

int, main, (,), {, int, a, =, 10, b, =, b, -, 30, ;, if,
(, ab,), return, (, b,), :, else, return, (, a,), ;, ?, void,
main, (,), {, int, x, =, 10, y, =, 20, ;, char, *, a, ;, a, =,
{, x, ;, x, =, Ixab, ;, ?

⇒ Total Tokens = 58

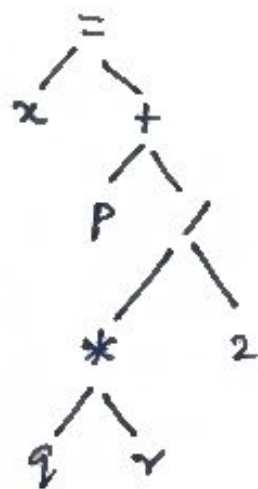
⇒ Lexical / Syntax Errors

- `b - 30;` - Minus - used instead of assignment = during variable declaration.
- `return(b):` - Colon : used instead of a semicolon ;
- `Ixab` - Invalid identifier name if I was intended to be an operator like ! , or an undeclared symbol token.

Question 4:- Processing `x = p + q * r / 2.`

Answer:-

- Lexical Analysis:- Evaluates input string using regex rules.
 - * Identifiers:- `x, p, q, r` matching `[a-zA-Z_][a-zA-Z0-9_]*`
 - * Constants:- `2` matching `[0-9]`
 - * Operators/punctuation:- `=, +, *, /`
- Parsing (precedence Rules: `*`, `/` over `+`, `=`):



- Intermediate Code:-
 - * `t1 = q * r`
 - * `t2 = t1 / 2`
 - * `t3 = p + t2`
 - * `x = t3`

Question 5:- Regular Expression over $\{0, 1\}$

Answer:-

1. Does not contain 110 as a substring:

$$(0|10)^*1^*$$

2. Contains both 101 and 010 as substrings:-

$$(0|1)^*(101(0|1)^*010|010|010(0|1)^*101|1010|0101)(0|1)^*$$

3. Both the numbers of 0s and 1s are odd:

- DFA with 4 steps.

$$(E_0 E_1, E_0 0_1, 0_0 E_1, 0_0 0_1)$$

gives $(00|11)^*(01|10)((00|11)|(01|10)(00|11)^*(01|10))^*$